



LALR(1) 语法分析生成器 — 使用说明书

一、软件概述

本程序是一个 LALR(1) 语法分析生成器，实现了以下功能：

- 文法编辑、保存和加载
- FIRST 集、FOLLOW 集的计算与展示
- LR(0) DFA 构建与展示
- SLR(1) 文法判定（含冲突原因显示）
- LR(1) DFA 构建与展示（含向前搜索符）
- LALR(1) DFA 构建与展示（先行符号传播法）
- LALR(1) ACTION + GOTO 分析表的生成与展示
- 输入句子的 LALR(1) 语法分析及过程追踪

二、运行方式

1. Qt 图形界面版：双击 LALR1_Visual.exe
2. 控制台版：双击 parse1.exe 或命令行运行

三、Qt 版界面操作说明

【功能1：文法编辑与加载】

操作：启动 → "文法"Tab → 编辑文本框中的文法 → 点击"加载文法"按钮

说明：默认已有示范文法。文法格式为每行一个产生式，

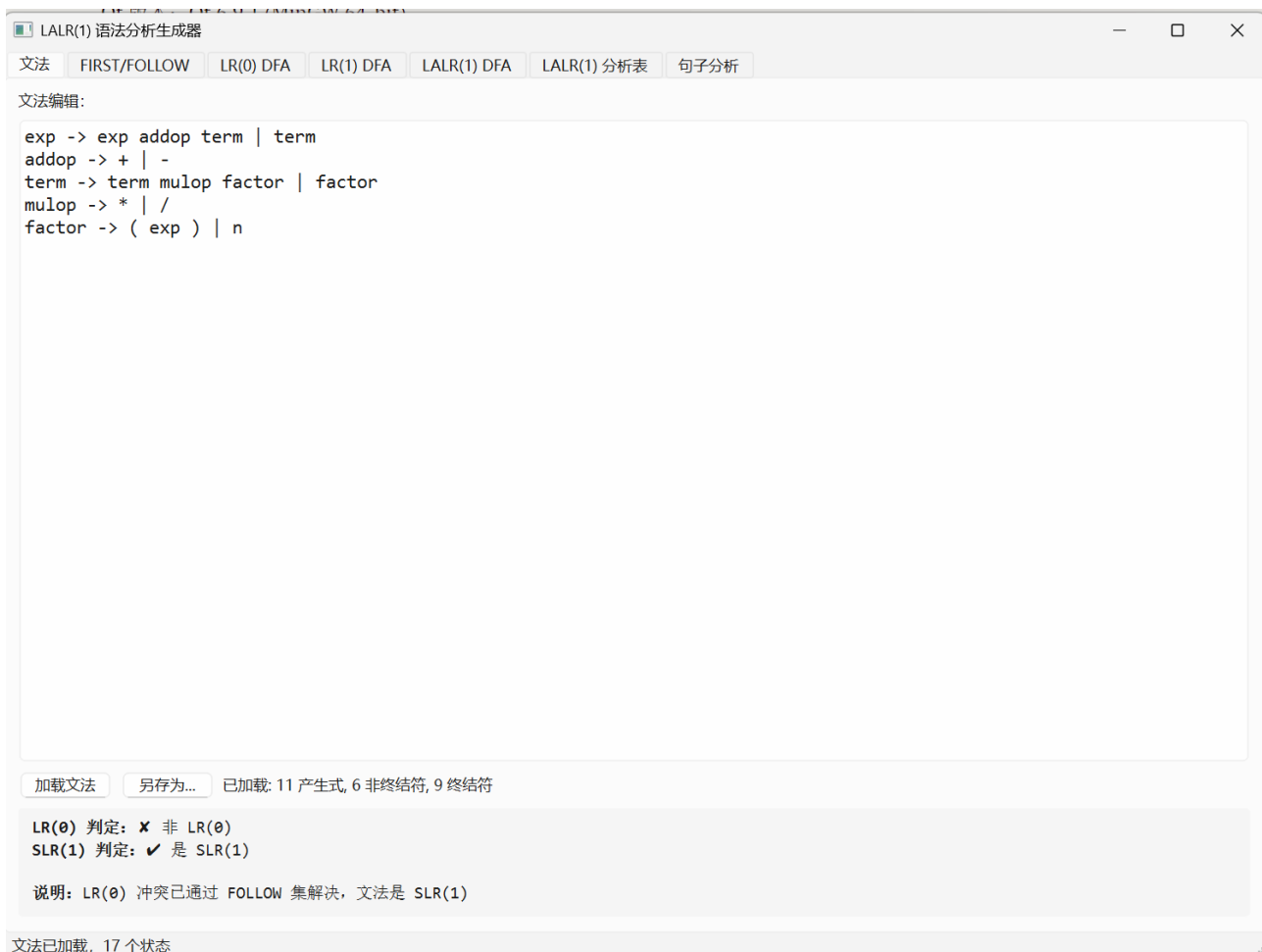
使用 -> 分隔左右部，| 表示分支。

示例：

$A \rightarrow (A) | a$

输出：下方状态栏显示"文法已加载，X 个状态"。

注意：文法中的符号之间用空格隔开，如 (A) 不能写成 (A)。



【功能2：查看 FIRST / FOLLOW 集】

操作：点击 "FIRST/FOLLOW" Tab

说明：以表格形式展示每个非终结符的 FIRST 集和 FOLLOW 集。

代表空串 ϵ ，\$ 代表输入结束标记。

LALR(1) 语法分析生成器

文法

FIRST/FOLLOW

LR(0) DFA

LR(1) DFA

LALR(1) DFA

LALR(1) 分析表

句子分析

非终结符	FIRST	FOLLOW
A	{ (, a }	{), \$ }
S'	{ (, a }	{ }

文法已加载, 7 个状态

【功能3：查看 LR(0) DFA】

操作：点击 "LR(0) DFA" Tab

说明：表格分三列：State(状态编号)、Items(项目集)、Transitions(转移边)。

圆点 · 表示当前分析位置。底部显示该文法是否为 LR(0) 和 SLR(1)。

LALR(1) 语法分析生成器		
文法	FIRST/FOLLOW	LR(0) DFA
		LR(1) DFA
		LALR(1) DFA
		LALR(1) 分析表
		句子分析
LR(0) DFA		
State	Items	Transitions
0	A -> . (A)...	A -> 1, (-> 2, a -> 3
1	S' -> A . \$...	\$ -> 4
2	A -> . (A)...	A -> 5, (-> 2, a -> 3
3	A -> a	
4	S' -> A \$	
5	A -> (A .)...	) -> 6
6	A -> (A)	
文法已加载, 7 个状态		

【功能4：查看 LR(1) DFA】

操作：点击 "LR(1) DFA" Tab

说明：与 LR(0) 类似，但每个项目后带有 [a] 向前搜索符。

LR(1) 的状态数多于 LR(0)。

LALR(1) 语法分析生成器

文法FIRST/FOLLOWLR(0) DFALR(1) DFALALR(1) DFALALR(1) 分析表句子分析

LR(1) DFA

State	Items	Transitions
0	A -> . (A) [\$]...	A → 1, (→ 2, a → 3
1	S' -> A . \$ [\$]...	\$ → 4
2	A -> . (A) []]...	A → 5, (→ 6, a → 7
3	A -> a . [\$]...	
4	S' -> A \$. [\$]...	
5	A -> (A .) [\$]...	) → 8
6	A -> . (A) []]...	A → 9, (→ 6, a → 7
7	A -> a . []]...	
8	A -> (A) . [\$]...	
9	A -> (A .) []]...	) → 10
10	A -> (A) . []]...	

文法已加载, 7 个状态

【功能5：查看 LALR(1) DFA】

操作：点击 "LALR(1) DFA" Tab

说明：以纯文本形式展示 LALR(1) DFA 的所有状态、项目和 lookahead 集合。

状态数与 LR(0) 相同（同心项目合并）。底部显示 LALR(1) 判定结果。

LALR(1) 语法分析生成器		
文法	FIRST/FOLLOW	LR(0) DFA
LR(1) DFA	LALR(1) DFA	LALR(1) 分析表
句子分析		
LALR(1) DFA		
State	Items	Transitions
0	A -> . (A) [\$]...	A -> 1, (-> 2, a -> 3
1	S' -> A . \$ [\$]...	\$ -> 4
2	A -> . (A) []]...	A -> 5, (-> 2, a -> 3
3	A -> a . [] \$]...	
4	S' -> A \$. [\$]...	
5	A -> (A .) [] \$]...	) -> 6
6	A -> (A) . [] \$]...	
文法已加载, 7 个状态		

【功能6：查看 LALR(1) 分析表】

操作：点击 "LALR(1) 分析表" Tab

说明：

- ACTION 表（上方）：状态行 × 终结符列
 - 蓝色背景 = 移进动作 sN（移进符号并转到状态 N）
 - 橙色背景 = 归约动作 rN（按产生式 N 归约）
 - 绿色背景 = Accept（接受输入）
- GOTO 表（下方）：状态行 × 非终结符列
 - 数字为归约后的转移目标状态

LALR(1) 语法分析生成器

文法

FIRST/FOLLOW

LR(0) DFA

LR(1) DFA

LALR(1) DFA

LALR(1) 分析表

句子分析

ACTION 表:

State	(	)	a	\$
0	s2		s3	
1				s4
2	s2		s3	
3		r1		r1
4				acc
5		s6		
6		r0		r0

GOTO 表:

State	A
0	1
2	5

文法已加载, 7 个状态

【功能7：句子分析】

操作：

1. 点击 "句子分析" Tab
2. 在输入框中输入要分析的句子，如 **a** 或 **(a)**
3. 点击"分析"按钮或按回车键

重要：输入句子时，每个 **token**（单词/符号）之间必须用空格隔开！

例如：**(a)** 不能写成 **(a)** 或 **a + a** 不能写成 **a+a**

输出：

下方表格显示完整的分析过程，每行包含：

Step（步骤）、**State Stack**（状态栈）、**Symbol Stack**（符号栈）、**Remaining Input**（剩余输入）、**Action**（动作说明）

结果以绿色 **✓** **ACCEPTED** 或红色 **✗** **REJECTED** 显示。

注意：点击"清除"可清空输入和结果。

LALR(1) 语法分析生成器				
文法	FIRST/FOLLOW	LR(0) DFA	LR(1) DFA	LALR(1) DFA
LALR(1) 分析表				
句子分析				
输入句子: ((a))				分析 清除 ✓ ACCEPTED
Step	State Stack	Symbol Stack	Remaining Input	Action
0	0		((a)) \$	
1	0 2	(	(a)) \$	Shift (→ 2
2	0 2 2	((	a)) \$	Shift (→ 2
3	0 2 2 3	((a	)) \$	Shift a → 3
4	0 2 2 5	((A	)) \$	Reduce by Rule 1: A → a
5	0 2 2 5 6	((A)	) \$	Shift) → 6
6	0 2 5	(A	) \$	Reduce by Rule 0: A → (A)
7	0 2 5 6	(A)	\$	Shift) → 6
8	0 1	A	\$	Reduce by Rule 0: A → (A)
9	0 1 4	A \$		Shift \$ → 4
10	0 1 4	A \$		Accept

四、控制台版操作说明

直接双击运行 `parse1.exe`，或通过命令行：

`echo 句子 | parse1.exe`

例如：`echo "a" | parse1.exe`

注意：交互式输入句子时，每个 **token** 之间必须用空格隔开。

例如输入 `( a )` 而非 `(a)`。

程序输出顺序：

1. 所有产生式
2. FIRST 集、FOLLOW 集
3. LR(0) DFA（状态 + 转移）
4. LR(1) DFA（状态 + 转移 + preview）
5. LALR(1) DFA（状态 + lookahead）
6. LALR(1) ACTION 表 + GOTO 表
7. 交互式句子分析提示

五、注意事项

1. 文法中的 ε （空串）用 # 表示
2. 程序的开始符号默认为第一个非终结符
3. 增广产生式 $S' \rightarrow S \$$ 由程序自动添加
4. Qt 版需确保同目录下有所有 .dll 文件
5. 控制台版退出输入 "quit" 或 "exit"